

エスカレシミュレーター 実装仕様書

1. システム概要

1.1 基本情報

項目	内容
システム名	エスカレシミュレーター
システム種別	ITヘルプデスク / カスタマーサポートの対応順シミュレーター
利用対象	申請者（問い合わせする人）、サポート担当者、管理者
主な機能	ユーザー認証、顧客、チケット起票、点数、待ち一覧、次を引き取る、緊急度の変更、対応完了、約束時間の警告、ダッシュボード、デモ
通信方式	REST API、WebSocket
データの本体	PostgreSQL（チケットの正しい状態はここ）
順番メモリと回数制限	Redis
画面への配信	サーバーが複数あるときは、Redis の Pub/Sub で他のサーバーへ知らせを渡す

1.2 システム目的

社内外の問い合わせを、緊急度・契約プラン・待ち時間から「今どれを先にやるか」の点数にまとめ、手が空いた担当者がその1件を引き取る。待ち順の変化は画面にすぐ反映する。

1.3 解決する課題

課題	解決方法
到着順だと重大障害が埋もれる	点数の式で緊急度を最優先にする
VIPと一般の扱いが人によって違う	同じ緊急度の中ではプランの点数で順を決める
安いプランが長く待たされ続ける	約束時間オーバーの件を、まだ時間内の高いプランより先にする
同じチケットが二人に渡る	順番メモリからいちばん点が高い1件を取り出して同時に消し、PostgreSQL では対応待ちのときだけ担当を付ける
緊急度を変えても並びがすぐ変わらない	点数をやり直して順番メモリを更新し、接続中の画面へ知らせる
約束時間を過ぎても気づかない	30秒ごとの処理でオーバーを見つけ、赤くして知らせる
誰が空いているか分からない	担当者の待機中 / 作業中 / 離席を保存し、画面へ届ける

1.4 要件定義で足したこと（実装でも変えない）

最初のたたき台に足りなかった点は、要件定義で次のとおり決めてある。実装仕様でも同じ言葉・同じ規則で書く。

足したこと	決めた内容
申請者と契約プランの結びつき	申請者は必ず1つの顧客に属する。起票画面でプランを手入力しない。所属顧客から取る。本番相当の申請者は所属が固定。デモでは管理者が付け替えてできる
社内からの問い合わせ	顧客区分は社内。契約プランは持たないが、点数上のプランは Enterprise と同じ 3。約束時間の初期値は15分
担当者の空き	次を引き取るのは待機中だけ。同時に持てる対応中は1件。作業中・離席はボタンを拒否する。離席または切断から5分で、対応中は対応待ちに戻す。約束時間の起算は作った時刻のまま
緊急度を変えられる人	担当者は自分が対応中の件だけ上げられる。下げは管理者だけ。完了した件は変えない
知らせの相手	待ち順の変化と「誰が引き取ったか」は担当者・管理者へ。申請者には自分のチケットだけ。現場全体の数字は管理者だけ
ダッシュボードの数字	待ち件数は対応待ち。約束時間は「初めて担当が付くまで」。平均の対応時間は引き取った時刻から完了まで

足したこと	決めた内容
デモ	管理者が件数と間隔を指定し、通常の起票と同じ道でダミーを流す
Token の失効	役割・所属顧客・アカウント状態を変えたら、その人の auth_version を上げる。Token の版がそれより古ければ、以降のリクエストを拒否する

1.5 技術スタック

要件定義書の技術スタックに加え、実装で固定する値は次とする。JWT は HS256、Password は bcrypt Cost 12、Refresh Token は crypto/rand と SHA-256、CSRF は Double Submit Cookie。フロントのテストは Vitest。

1.6 このアプリでの狙い

狙い	内容	主に使うもの
点数	緊急度・プラン・約束時間オーバー・待ち時間の式と判定表。保存先を知らないルールだけが計算する	Go の Entity (<code>priority_score.go</code>)、Go のテスト
1対応1案件	待機中の担当者が、今いちばん先の1件だけを取る。同時押しでも同じ件は一人にしか渡らない	Redis の順番メモリ、PostgreSQL、Go の UseCase が両方を組む
待ち一覧がすぐ動く	起票・引き取り・緊急度の変更・約束時間オーバーで、順位と赤がすぐ変わる	WebSocket、30秒ごとの定期処理、順番メモリ

2. プログラムの分け方

2.1 処理の順

```

入口 (Router)
↓
共通処理 (認証、CSRF、回数制限、ログ)
↓
受け口 (Controller)
↓
業務の手順 (UseCase)
↓
保存 / 順番メモリ / 画面への知らせ
↓
PostgreSQL / Redis / 接続中の画面

```

画面の即時更新 (WebSocket) は、つなぐときに認証し、役割に応じた待ち一覧一式を送る。

2.2 各層の役割

層	役割
Router	URL と受け口を結びつける
Middleware	認証、CSRF、CORS、回数制限、Request ID、ログ、Panic の捕捉
Controller	入力を受け取り、業務の手順を呼び、結果を返す
UseCase	順番、誰がやってよいか、Transaction、記録。COMMIT のあとで順番メモリと知らせ
Entity	データの形と、点数の計算。保存先を知らない
Repository	PostgreSQL への読み書き
Redis	順番メモリ、回数制限、Access Token の失効、接続、デモの進行
配信	複数の API サーバーの間で、画面への知らせを Pub/Sub で受け渡す
WebSocket	接続、役割ごとの知らせ、強制切断
定期処理	点数のやり直し、約束時間オーバーの検知、切断後の担当はなし

点数の計算ファイルは式だけを持つ。順番メモリでやり直すか、データベースで計算するかは知らない。順番メモリと30秒ごとの処理がこの計算を呼ぶ。

2.3 やってはいけないこと

層	禁止
Router	権限の判定、DB 更新
Controller	保存への直接アクセス、対応待ちから対応中への変更、Transaction、Token の直接発行
点数の計算	Redis、GORM、HTTP を import する

2.4 業務の手順の基本順

1. 入力確かめる
2. 実行する人を取り、利用中であることを確かめる
3. 役割と、そのチケット・顧客を触ってよいかを確かめる
4. 業務の条件を確かめる
5. Transaction を始める
6. PostgreSQL を更新する
7. 操作の記録を残す
8. COMMIT する
9. 順番メモリを更新する
10. 画面へ知らせる
11. 結果を返す

COMMIT の前に知らせを送らない。COMMIT が失敗したら順番メモリも知らせもしない。知らせの失敗ではデータベースを戻さない。

「次を引き取る」だけは例外で、順番メモリからの取り出しをデータベース更新より先に行う。失敗したときの戻し方は当該手順に書く。

2.5 ディレクトリ

- 点数・チケット・顧客・担当者の形 → `entity/` (点数は `priority_score.go`)
- 起票、次を引き取る、緊急度の変更、完了、稼働、約束時間の確認、デモ → `usecase/`
- PostgreSQL → `repository/`
- 順番メモリ → `redis/ticket_order.go`
- 配信 → `redis/notifier.go`
- 画面への知らせ → `websocket/`
- HTTP の入口 → `controller/` `router/`

フロントは申請者・担当者・管理者で `pages/` と `components/` を分ける。

3. 設定

環境変数	必須	初期値	内容
JWT_SECRET	必須	なし	32バイト以上。無いと起動しない
JWT_ISSUER	任意	escalator-api	
JWT_AUDIENCE	任意	escalator-client	
ACCESS_TOKEN_TTL_MINUTES	任意	15	
REFRESH_TOKEN_TTL_MINUTES	開発	30	
REFRESH_TOKEN_TTL_DAYS	本番	14	

環境変数	必須	初期値	内容
BCRYPT_COST	任意	12	
COOKIE_SECURE	本番	true	
REFRESH_TOKEN_COOKIE_NAME	任意	escalator_refresh_token	
CSRF_TOKEN_COOKIE_NAME	任意	escalator_csrf_token	
LOGIN_MAX_FAILURES	任意	5	
LOGIN_LOCK_SECONDS	任意	開発900 / 本番3600	
PRIORITY_RECALC_SECONDS	任意	30	点数をやり直す間隔
AGENT_DISCONNECT_GRACE_SECONDS	任意	300	切れてから担当を外すまでの秒
CORS_ALLOWED_ORIGIN	必須	なし	フロントの Origin
REDIS_PUBSUB_CHANNEL_PREFIX	任意	escalator:notify:	複数サーバーで配信に使うチャンネル名の接頭辞

4. 認証

- Access Token は JWT、15分、ブラウザのメモリのみ。 `Authorization: Bearer`
- Token に入れるのは User ID、発行日時、有効期限、認証の版。役割は入れない。毎回データベースの役割を正とする
- 毎回、認証の版が今の auth_version と一致するかも確かめる。古ければ拒否する
- 再ログイン用の印は JavaScript から読めない Cookie。データベースには Hash だけ。使うたびに新しい印を出し、古い印は使えない
- 無効な印の再利用は、同じログインの系統をすべて無効にし、画面の常時接続を切る
- CSRF はまず Sec-Fetch-Site を見る。same-origin でなければ拒否する
- Sec-Fetch-Site が付いていないとき（古いブラウザなど）は、ダブルサブミットで確かめる。Cookie とヘッダーの CSRF 値が一致しなければ拒否する
- この確認は再発行とログアウトだけ必須
- Password は 8～15文字、英字と数字を各1以上、bcrypt。平文は保存もログも Response も禁止
- ログイン失敗のメッセージは、User がいない・Password 違い・停止中・削除済みを区別しない
- 会員登録の最初の役割は申請者。所属顧客は空。起票するには管理者が顧客を付ける。サンプルデータでは最初から付けておく
- 役割・所属顧客・アカウント状態を変えたときは、auth_version を1つ上げる。以降そのユーザーの古い Token はすべて拒否される

5. データの形

5.1 ユーザー

画面の言葉	列	内容
	id	UUID
メール	email	一意
	password_hash	bcrypt
表示名	name	1～50文字
役割	role	申請者 / 担当者 / 管理者
所属顧客	customer_id	申請者は必須。担当者・管理者は空
アカウント状態	status	利用中 / 停止 / 削除
	auth_version	Token をまとめて無効にする番号

役割・状態・所属顧客の列は、保存はローマ字の決まった語（applicant / agent / admin、active / suspended / deleted）とし、画面ではこの表の日本語で表示する。

1アカウントは1役割。変更は管理者だけ。担当者を申請者にするとき、対応中があれば先に待ち一覧へ戻す。

申請者に顧客が無い更新、担当者・管理者に顧客を付ける更新は拒否する。

停止と削除の段階は利用中→停止→削除。削除時はメールを匿名化する。実行は管理者。

5.2 顧客

画面の言葉	列	内容
顧客名	name	1～100文字
区分	kind	社外 / 社内
契約プラン	plan	Free / Pro / Enterprise
初回対応の約束時間（分）	sla_minutes	1～1440

区分	説明	プランの点数	約束時間の初期値
社外 Free	無償	1	120分
社外 Pro	有償	2	60分
社外 Enterprise	VIP / 大口	3	15分
社内	社員・協力会社。契約プランは持たない	3（Enterprise と同じ）	15分

区分・契約プランの列も、保存はローマ字の決まった語（external / internal、free / pro / enterprise）とし、画面ではこの表の日本語で表示する。

社内のとき保存上の plan はいつも enterprise にする。起票画面ではプランを出さない。

プランの点数は顧客のメソッドが返す。順番メモリを知らない。

5.3 担当者の稼働

担当者アカウントを作ったときに1行作る。申請者・管理者には作らない。

画面	保存名	意味
待機中	available	次を引き取ることができる
作業中	busy	対応中を1件持っている
離席	offline	次を引き取らない

作業中への手動変更は不可。作業中になるのは次を引き取るが成功したときだけ。待機中と離席は本人が切り替えられる。管理者は任意の担当者を離席にできる。

5.4 チケット

画面の言葉	列	内容
件名	title	1～200文字
詳細	description	1～5000文字
種類	category	障害 / 不具合 / 質問 / 要望 / その他
緊急度	severity	1～4
状態	status	対応待ち / 対応中 / 完了
顧客	customer_id	起票時の所属。必須
申請者	created_by	
担当者	assigned_agent_id	空でも可
点数	priority_score	最後に計算した値
約束時間オーバーになった時刻	sla_violated_at	空でも可

画面の言葉	列	内容
知らせを出した時刻	sla_notified_at	空でも可。1チケット1回
作った時刻	created_at	約束時間の起算。緊急度を変えても変えない
引き取った時刻	claimed_at	待ちに戻すと空
完了した時刻	closed_at	空でも可

種類・状態の列も、保存はローマ字の決まった語（incident / bug / question / request / other、open / in_progress / closed）とし、画面ではこの表の日本語で表示する。

緊急度: 4 システム全停止、3 一部機能不可、2 不具合報告、1 質問・要望。

完了を元に戻す機能は作らない。間違えたら管理者が新しい件として出し直す。

対応メモ、緊急度の変更履歴、デモの実行、再ログイン用の印、操作の記録は要件定義書のデータモデルどおり。

6. 点数の計算

ファイル: `entity/priority_score.go`

入力: 緊急度、プランの点数、作った時刻、今の時刻、約束時間（分）、すでに担当が付いているか。

すでに担当が付いているときは、呼び出し側が順番メモリから外す。この関数は点数だけ返す。

```
待ち分数 = 作ってから今までの分数（秒は切り捨て）。99を超えたら99

約束時間オーバー = まだ担当が付いていない かつ 待ち分数が約束時間以上 なら 1、それ以外 0

点数 =
    緊急度 × 1_000_000
    + 約束時間オーバーなら 10_000
    + プランの点数 × 100
    + 待ち分数
```

緊急度が1〜4以外、プランの点数が1〜3以外、約束時間が1未満ならエラー。

先にやる順番（要件定義と同じ）

1. 緊急度がいちばん強い。緊急度が1違うと、待ち時間やプランでは追い越せない
2. 緊急度が同じなら、約束時間を過ぎた件を、まだ時間内の高いプランより先にする。Free が約束時間を過ぎているのに Enterprise の新しい件を先取ることはしない
3. 緊急度が同じで、どちらも時間内（またはどちらもオーバー）なら、プランが高い方を先にする
4. それでも同じなら、待っている時間が長い方を先にする
5. 待ち時間の加点は最大99。時間内ではプランの差（100）を追い越せない。プランの差を超えるのは約束時間オーバーの加点だけ

やらないこと: 画面表示のときだけ点数を出し、順番メモリには到着順だけ置く。待ち時間だけで Enterprise の新規を追い越す。

待ち一覧を返すとき、点数が同じなら先に作られたチケットを先にする。作った時刻も同じなら ID が小さい方。順番メモリ上の点数が同じでも、画面の順はこの規則で決める。

6.1 判定表

ケース	A	B	先
緊急度が違う	緊急度4 / Free / 待ち0分	緊急度3 / Enterprise / 待ち99分	A
約束時間オーバー vs 高いプランの新規	緊急度2 / Free / 待ち120分（約束120分でオーバー）	緊急度2 / Enterprise / 待ち0分（約束15分でまだ時間内）	A
同じ緊急度・どちらも時間内	緊急度3 / Enterprise / 待ち1分	緊急度3 / Free / 待ち10分	A
同じ緊急度・どちらもオーバー	緊急度2 / Pro / 待ち200分	緊急度2 / Free / 待ち200分	A（プランが高い方）

ケース	A	B	先
待ちだけ違う	緊急度1 / Pro / 待ち50分	緊急度1 / Pro / 待ち10分	A
社内 vs Enterprise	緊急度3 / 社内 / 待ち0分	緊急度3 / Enterprise / 待ち0分	同じ点。先に作られた方
社内の質問 vs 社外の全停止	緊急度1 / 社内	緊急度4 / Free	B

6.2 点数をいつやり直すか

タイミング	処理
チケット作成	点数を出し、PostgreSQL に保存し、順番メモリにも載せる
緊急度の変更	やり直して、データベースと順番メモリを更新する（対応待ちのとき順番メモリも）
30秒ごと	対応待ちの点数をやり直して順番メモリを更新する。待ち時間と約束時間オーバーが変わるため
次を引き取る / 完了	順番メモリから外す。やり直しは不要
待ちに戻ったとき	今の時刻でやり直して順番メモリに載せる

30秒は、いちばん短い約束時間（15分）より十分短い。

7. 順番メモリ

項目	内容
何をするか	対応待ちの今の順番を Redis に持たせる。点数が高いほど先
Key	<code>queue:tickets</code>
点数	ルール計算が出した点数
値	チケットの ID

操作	いつ
並びへ載せる	作成、やり直し、緊急度変更（対応待ち）、待ちへ戻ったとき
いちばん点が高い1件を取り出して同時に消す	次を引き取る。途中で他の担当者の操作は割り込まない
並びから外す	完了。対応中になったあと残っていれば外す

チケットの状態の正は PostgreSQL。順番メモリは速く出すため。起動時と食い違いを見つけたときは、対応待ちから順番を作り直す。

順番メモリが使えないとき、「次を引き取る」は受け付けない（サービス一時利用不可）。待ち一覧の GET は PostgreSQL の対応待ちを、点数が高い順・同じ点なら先に作られた順で返してよい。

クライアントが「読んでから消す」を2往復してはいけない。

7.1 配信

項目	内容
何をするか	API サーバーが複数台のとき、WebSocket をつないでいないサーバーで起きた更新を、つないでいるサーバーへ届ける
チャンネル	<code>escalator:notify:{role}</code>
発行	UseCase が COMMIT のあと、順番メモリの更新と同じところで発行する
購読	各サーバーが起動時に購読し、自分がつないでいる画面へ配る
サーバーが1台のとき	配信を経由せず、そのまま画面へ知らせてよい

8. 誰が何をしてよいか

毎回、データベースの役割とアカウント状態を見る。

操作	申請者	担当者	管理者
自分のチケット一覧・詳細	可（自分が起票した件）	—	可
チケット起票	可（所属顧客必須）	不可	不可。デモは中から同じ起票手順を呼ぶ
待ち一覧	不可	可	可
次を引き取る	不可	可（待機中かつ対応中0件）	不可
対応メモ	不可	自分の対応中	可
対応完了	不可	自分の対応中	可
緊急度を上げる	不可	自分が対応中だけ	対応待ち・対応中
緊急度を下げる	不可	不可	対応待ち・対応中
担当を外す	不可	離席にしたとき自動	可
顧客の登録・更新	不可	不可	可
利用者の役割・所属顧客	不可	不可	可
ダッシュボード	不可	不可	可
デモ	不可	不可	可

担当者の詳細: 待ち一覧にある対応待ちは見られる。対応中と完了は、自分が担当した件、または今の担当が自分。他人の対応中は拒否。無い件と、申請者から見た他人の件は、どちらも「見つからない」にする（存在を漏らさない）。

9. 業務の手順

9.1 チケット起票

目的: 新しい問い合わせを作り、待ち一覧に載せる。

入力: 実行する人、件名、詳細、種類、緊急度。顧客IDは申請者の所属から取る。手入力しない。

事前: 役割が申請者、利用中、所属顧客がある。無ければ「所属顧客が設定されていません」。

流れ:

1. 所属顧客を取る（プランと約束時間）
2. 今の時刻で点数を出す
3. 対応待ちで保存する。担当者は空
4. COMMIT
5. 順番メモリに載せる
6. 「チケットが作られた」を知らせる。担当者・管理者は全員。申請者は自分が出した件だけ

順番メモリへの載せに失敗しても起票は成功とし、次の30秒のやり直しで載せる。失敗はログに残す。

社内の申請者は社内枠の顧客に属していること。プランは手入力させないので、社内も自動でプランの点数 3、約束時間は顧客の値（初期15分）になる。

9.2 申請者の自分のチケット一覧

自分が起票した件だけ。更新が新しい順。他人の件や全体の待ち一覧は出さない。

出すもの: 件名、緊急度、状態、作成日時、約束時間までの残りまたはオーバー、担当者名（付いたあと）。

9.3 待ち一覧

担当者または管理者。対応待ちを点数が高い順、同じ点なら先に作られた順。

出すもの: 順位、件名、緊急度、プラン、待ち時間、点数、約束時間までの残りまたはオーバー、顧客名。詳細本文は出さない。

約束時間オーバーは赤。

9.4 チケット詳細

要件定義の「誰が見られるか」とおり。コメントは古い順。緊急度の変更履歴も出す。

9.5 次を引き取る

目的: 今いちばん先の対応待ちを、ボタンを押した担当者へ渡す。同時押しでも一人にしか渡さない。

事前: 役割が担当者、稼働が待機中、対応中を持っていない。作業中・離席は「待機中のときだけ引き取れます」。すでに対応中があれば「対応中のチケットがあります」。

流れ:

1. 稼働を行ロックし、待機中を確かめる
2. 対応中が0件であることを確かめる
3. 順番メモリから、点数がいちばん高いチケットを1件だけ取り出し、同時に並びから消す。二人が同時でも同じ ID は一人にしか渡らない
4. 待ちが0件なら「待ちチケットがありません」
5. PostgreSQL で「まだ対応待ちなら」対応中にし、担当者と引き取った時刻を書く
6. 1件更新できたら、担当者を作業中にする。記録を残す。COMMIT。「チケットが引き取られた」を知らせる
7. 更新できなかった（すでに完了など）ときは、その ID は捨て、次に点が高い件で同じことを繰り返す（最大3回）
8. データベースの書き込み自体が失敗したら、取り出した ID を同じ点数で順番メモリへ戻す。担当者は待機中のまま
9. 3回とも更新できなかったときは、順番メモリの食い違いとみなし「待ち順を一時的に利用できません」を返す。担当者は待機中のまま。取り出した ID は順番メモリへ戻さない

管理者は次を引き取らない。問い合わせの登録もしない。

9.6 対応完了

入力: 終了コメント（必須、1～2000文字）。

条件: 対応中であること。実行者はその件の担当者、または管理者。対応待ちのままでは完了にできない。

完了にした時刻を入れる。対応時間は引き取った時刻から完了した時刻。担当者本人が閉じたら待機中に戻す。管理者が閉じ、担当が別人ならその担当者も待機中に戻す（1人1件の前提）。順番メモリから外す。「チケットが完了した」を知らせる。

9.7 緊急度の変更

入力: 新しい緊急度、理由（必須、1～500文字）。

権限は要件定義の表。同じ値への変更は入力エラー。完了は不可。

対応待ちなら点数をやり直して順番メモリを更新する。対応中なら履歴と点数列だけ更新する（順番メモリには載っていない）。

約束時間の起算（作った時刻）は変えない。

「緊急度が変わった」を知らせる。申請者は自分の件のときだけ。

9.8 対応メモ

対応中のメモ。担当者は自分の対応中のみ。管理者は対応待ち・対応中。完了へのメモは最初の版では作らない。

9.9 担当を外して待ち一覧へ戻す

管理者が明示するとき、本人が離席にしたとき、切れてから5分戻らないときに使う。

対応中を対応待ちに戻す。担当者と引き取った時刻は空にする。約束時間オーバーの印は消さない（遅れを帳消ししない）。今の時刻で点数を出して順番メモリに載せる。離席操作なら稼働は離席。管理者の解除で担当を空けるだけなら、その担当者は待機中。「チケットが待ちに戻った」を知らせる。

9.10 稼働状態

本人は待機中または離席だけ指定できる。作業中の指定は拒否。

離席へ変えるとき対応中があれば、待ち一覧へ戻す。

管理者は任意の担当者を離席にできる。

「担当者の稼働が変わった」を、待ち一覧を見ている担当者・管理者へ知らせる。

9.11 顧客の登録・更新

管理者。社内ならプランは Enterprise 相当に固定。約束時間は分で 1~1440。

プランや約束時間の変更は、その後の新規起票と、対応待ちの30秒やり直しから効く。

9.12 申請者と顧客の結びつき（利用者管理）

管理者。申請者にするときは所属顧客必須。担当者にするときは稼働の行を無ければ待機中で作り、所属顧客は空。管理者も所属顧客は空。

デモでは、申請者の所属顧客を管理者が付け替えてよい。本番相当の申請者は所属が固定、という要件を、画面では「デモ用の付け替え」として管理者画面に置く。

役割・所属顧客・アカウント状態のいずれかを変えたときは、4章の規則どおり対象ユーザーの auth_version を1つ上げる。

9.13 ダッシュボード

管理者だけ。申請者にも担当者にも現場全体の数字は出さない。担当者画面は自分の状態と待ち一覧だけ。

数字	定義（要件定義と同じ）
待ち件数	対応待ちの件数
対応中件数	対応中の件数
平均の対応時間	直近24時間に完了した件の、最後に引き取った時刻から完了した時刻までの平均。途中で離席して別の人が引き取った場合は、最後の引き取り時刻を使う
平均の初回応答	直近24時間に初めて担当が付いた件の、作った時刻から引き取った時刻までの平均
今の約束時間オーバー	対応待ちで、作ってから約束時間以上経っている件数
24時間の約束時間オーバー	直近24時間に、担当が付いた時点ですでにオーバーしていた件数
担当者の空き	待機中 / 作業中 / 離席の人数

最初は通常の取得。その後は関連する知らせと30秒ごとのまとめ。

9.14 全件の監視

管理者。状態・緊急度・顧客・担当で絞り込む。条件に合う件が一覧になる。

9.15 デモ

管理者だけ。すでに実行中なら「デモはすでに実行中です」。

入力: 作る件数 1~50、間隔秒 1~10、緊急度とプランの出方（均等 / 障害多め）。

サンプル顧客を無作為に選び、ダミー件名で通常の起票と同じ道を通す。生成中でも、すでにある件の次を引き取る・完了は普通どおり。管理者が止める。件数に達したら終わる。

目的: 待ち順の変化、赤い警告、複数担当者の早い者勝ちを画面で見せる。

10. 画面の即時更新

接続 URL: [/ws](#)。つながるときに Access Token を確かめる。本番は暗号化必須。

役割	つながった直後
申請者	送らない（自分の一覧は通常の取得）
担当者	今の待ち一覧一式と、担当者の空き
管理者	今の待ち一覧一式と、ダッシュボードの今の数字

知らせ	申請者	担当者	管理者	内容
チケットが作られた	自分が出した件だけ	全員	全員	新しい件。待ち一覧に追加
チケットが引き取られた	自分の件が引き取られたとき	全員	全員	誰がどの件を引き取ったか
緊急度が変わった	自分の件のとき	全員	全員	新しい緊急度と点数
チケットが完了した	自分の件のとき	全員	全員	対応完了
チケットが待ちに戻った	自分の件のとき	全員	全員	離席などで担当が外れた
今の待ち一覧一式	送らない	つながった本人	つながった本人	接続した瞬間の待ち一覧と担当者稼働
約束時間オーバー	自分の件がオーバーしたとき	全員	全員	赤い警告
ダッシュボード更新	送らない	送らない	全員	数字のやり直し
デモの進み	送らない	送らなくてよい	実行した管理者	何件まで作ったか

全員に送ってよいのは、待ち順の変化と「誰が引き取ったか」である。申請者には、他の顧客の待ち一覧・現場全体の数字・他人の本文を送らない。担当者・管理者向けの待ち一覧の知らせにも詳細本文は入れない。

切れたときは一定間隔でつなぎ直し、成功したら今の待ち一覧をもう一度取る。担当者の接続がすべて切れてから5分戻らなければ、離席と同じく担当を外して待ち一覧に戻す。

アカウント停止時はその人の接続を切る。

11. 約束時間の監視（定期処理）

対応待ちのまま、初めて担当が付くまでの時間が約束を超えたことを見つける。Redis の期限切れだけに頼らない（待ち一覧の色とずれるため）。

30秒ごとに:

1. 対応待ちの点数をやり直す
2. 順番メモリを更新する
3. まだ知らせていないオーバーに印を付け、担当者と管理者へ知らせる。申請者にはその件の本人だけ
4. 担当者が切れたまま5分なら、担当を外す

サーバー起動時に、対応待ちから順番メモリを1回作り直す。

多重実行しないよう、定期処理用の短いロックを付ける。

12. 画面遷移とログイン状態の共有

項目	内容
公開	<code>/login</code> 、 <code>/register</code>
申請者	<code>/tickets</code> 、 <code>/tickets/new</code> 、 <code>/tickets/:ticketId</code>
担当者	<code>/agent/queue</code> 、 <code>/agent/tickets/:ticketId</code>
管理者	<code>/admin/dashboard</code> 、 <code>/admin/tickets</code> 、 <code>/admin/tickets/:ticketId</code> 、 <code>/admin/customers</code> 、 <code>/admin/users</code> 、 <code>/admin/demo</code>
未ログイン	ログインが必要な URL はログイン画面へ
役割が違う	自分のホームへ戻る。API も拒否
ログイン済みのログイン画面	役割ごとのホームへ

申請者ホームは `/tickets`、担当者は `/agent/queue`、管理者は `/admin/dashboard`。

ログイン状態の共有: ログイン中の人、役割、所属顧客、確認中か。再読み込みしても残る。ログアウトで消してログイン画面へ。Access Token はメモリ。再ログイン用の印は Cookie。

次を引き取るボタンは待機中だけ押せる。成功したら担当中の詳細へ。約束時間オーバーの行は赤。

13. API

API	内容
POST /api/auth/register	会員登録（申請者）
POST /api/auth/login	ログイン
POST /api/auth/logout	ログアウト
POST /api/auth/refresh	Access Token 再発行
GET /api/me	今のひとと役割、所属顧客
GET/POST/PATCH /api/customers	顧客（管理者）
GET/PATCH /api/users	利用者（管理者）
POST /api/tickets	起票（申請者）
GET /api/tickets	自分の一覧（申請者）
GET /api/tickets/:id	詳細（見られる人だけ）
GET /api/queue	待ち一覧（担当者・管理者）
POST /api/queue/claim	次を引き取る（担当者）
POST /api/tickets/:id/escalate	緊急度の変更
POST /api/tickets/:id/comments	対応メモ
POST /api/tickets/:id/close	対応完了
POST /api/tickets/:id/release	管理者が担当を外し、待ち一覧へ戻す
PATCH /api/agent/status	稼働状態
GET /api/admin/dashboard	現場の数字（管理者）
GET /api/admin/tickets	全件の監視（管理者）
POST /api/admin/demo/start	デモ開始
POST /api/admin/demo/stop	デモ停止
GET /api/health	稼働確認
GET /ws	役割ごとの即時更新

回数制限の対象と目安は要件定義書と同じ（ログイン、会員登録、再発行、起票、次を引き取る、緊急度変更、デモ開始）。超えたら 429。業務の手順の前で止める。

14. エラー

利用者向けには内部情報を出さない。

とき	HTTP	メッセージ
入力が条件を満たさない	400	入力内容を確認してください
認証できない	401	認証に失敗しました
役割が違う、他人の操作	403	この操作を実行できません
CSRF	403	リクエストを確認できません
チケットが無い、申請者から見た他人の件	404	チケットが見つかりません
所属顧客が無い	409	所属顧客が設定されていません
待機中ではない	409	待機中のときだけ引き取れます
すでに対応中がある	409	対応中のチケットがあります
待ちが0件	409	待ちチケットがありません
対応待ちのまま完了しようとした	409	対応中のチケットではありません
デモが実行中	409	デモはすでに実行中です
回数超過	429	時間を空けて再度お試しください

とき	HTTP	メッセージ
順番メモリが使えない	503	待ち順を一時的に利用できません
次を引き取るで3回とも更新できない	503	待ち順を一時的に利用できません

15. 入力と表示の安全

件名 1~200、詳細 1~5000、終了コメント 1~2000、理由 1~500。緊急度・種類・状態・プラン・稼働は決められた値だけ。クライアントの確認に加え、サーバーでも必ず確認する。

件名、詳細、コメント、顧客名、表示名は、そのまま HTML として埋め込まない。React の通常の文字表示を使う。

16. サンプルデータ

起動時または開発用投入:

- 顧客: 社内1、社外 Free / Pro / Enterprise 各1
- 管理者1、担当者2（待機中）、申請者（各プランと社内に1人ずつ、所属済み）

Password の平文は Git に置かない。

17. テスト（完了条件）

対象	確認
点数	判定表の全行
次を引き取る	2人が同時に押しても同じチケットが二人に付かない
対応待ちでない件	担当にならない
起票	順番メモリに載る。プランは所属顧客から付く。手入力していない
社内起票	プランの点数が 3、約束時間が社内顧客の値
緊急度を上げる	対応待ちなら待ち一覧の位置がすぐ変わる
担当者が緊急度を下げる	拒否
約束時間オーバー	加点され、まだ時間内の高いプランより先
待ちに戻す	引き取った時刻は空、作った時刻は変わらない、順番メモリに載る
離席	対応中が待ち一覧に戻る
申請者が待ち一覧	拒否
申請者が他人の詳細	見つからない
役割・状態の変更	変えた直後、古い Token での呼び出しが拒否される
複数サーバーでの配信	サーバーAで起きた知らせが、サーバーBにつながっている画面にも届く
ダッシュボード	申請者・担当者は拒否。数字の定義どおり
デモ	通常の起票と同じ知らせが飛ぶ

手動: デモで待ち一覧が動き、約束時間オーバーが赤くなり、二人同時の次を引き取るで二重にならない。